

SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY

COMPUTER SCIENCE & INFORMATION TECHNOLOGY DEPARTMENT

Fall 2024

Web Engineering (CS311T)

Assignment # 2 Sec C

Semester: 5th

Announced Date: Nov 30, 2024

Total Marks: 10

Instructor Name: Mohsin Ali

Batches: 2022F, 2020F

Due Date: Dec 07, 2024

Marks Obtained:

CLO #	Course Learning Outcomes (CLOs)	PLO Mapping	Bloom’s Taxonomy
CLO 2	Describe the following:	PLO_2 (Knowledge for solving Computing Problems)	C2 (Understanding)

Question 1:

Discuss four types of APIs.

Question 2:

Explain the terms Model, View and Controller in Model View Framework

Instructions:

- 1. Complete the assignment in groups, each group must have maximum 4 students.
- 2. Assignment should be submitted in the form of hard copy.
- 3. Copying is not allowed. Any similar assignments will get zero marks.
- 4. Insert this page on top of your assignment.

Group Members

Name	Roll Number
Saad Ali	2022F-BCS-162
Muzamil	2022F-BCS-316
Muhammad Faisal	2022F-BCS-152
Afnan	2022F-BCS-168

Question 1: Discuss four types of APIs.

APIs (Application Programming Interfaces) are mechanisms that allow software applications to communicate and share data or functionality. Here are four common types of APIs explained in detail:

1. Open APIs (Public APIs)

- **Definition:** Open APIs are publicly available and can be accessed by any developer. They are designed to provide access to specific services or data from an organization.
- **Key Features:**
 - No strict usage restrictions.
 - Often used to promote business by increasing third-party integrations.
 - Authentication and rate limits may still apply.
- **Examples:**
 - Google Maps API for embedding maps.
 - Twitter API for accessing tweets or user data.
- **Use Case:** A developer can integrate Google Maps API into an e-commerce platform to show store locations and provide directions.

2. Partner APIs

- **Definition:** Partner APIs are shared with specific business partners and are not publicly available. They require explicit permission and are used to facilitate collaboration between organizations.
- **Key Features:**
 - Enhanced security and control.
 - Restricted to trusted partners.
 - Focused on business-to-business (B2B) interactions.
- **Examples:**
 - Payment gateway APIs (e.g., PayPal Partner API) offered to specific e-commerce platforms.
 - Shipping APIs provided by logistics companies like FedEx or UPS to their partners.
- **Use Case:** A logistics company can use a partner API to integrate tracking and shipping features into a retailer's website.

3. Internal APIs (Private APIs)

- **Definition:** Internal APIs are designed for use within an organization. They allow different systems, teams, or services within the same company to communicate with each other.
- **Key Features:**
 - Restricted access for internal operations.
 - Focused on optimizing internal workflows and system integration.
 - Not exposed to external users.
- **Examples:**
 - A company's HR API for managing employee records internally.
 - APIs connecting different microservices in a distributed system.
- **Use Case:** An e-commerce company might use an internal API to integrate its inventory management system with its order-processing system.

4. Composite APIs

- **Definition:** Composite APIs combine multiple APIs into a single call, enabling the client to access data or services from different sources with minimal requests.
- **Key Features:**
 - Reduces the number of API calls.
 - Optimizes performance for complex operations.
 - Commonly used in microservices architectures.
- **Examples:**
 - A travel booking API that combines flight, hotel, and car rental services into a single response.
 - APIs in banking applications that combine account details, transaction history, and loan data in one request.
- **Use Case:** A travel website can use a composite API to fetch data about available flights, hotels, and cars in one go, enhancing user experience and reducing latency.

These four types of APIs cater to different use cases and audiences, helping organizations achieve various goals like improving collaboration, enabling integrations, and streamlining internal workflows.

Question 2: Explain the terms Model, View and Controller in Model View Framework

The **Model-View-Controller (MVC)** framework is an architectural pattern used in software development to separate concerns and organize code in a way that enhances maintainability, scalability, and testability. Here's a detailed explanation of its three primary components:

1. Model

Definition: The **Model** represents the data, business logic, and rules of the application. It is responsible for managing the state of the application and interacting with the database or other data sources.

- **Key Responsibilities:**
 - Retrieve and store data.
 - Apply business rules or validation logic.
 - Notify the **View** when there is a change in data (in frameworks with data-binding or observer patterns).
- **Characteristics:**
 - Independent of the user interface (UI).
 - Central to the application's functionality and core logic.
- **Example:**

In an e-commerce application, the **Model** for a product might contain attributes like ProductID, Name, Price, Description, and methods like CalculateDiscount() or CheckInventory().

2. View

Definition: The **View** is responsible for displaying the data to the user and providing the user interface. It retrieves data from the **Model** and renders it in a user-friendly format.

- **Key Responsibilities:**
 - Present data to the user.
 - Reflect changes in the **Model** (in real-time if the framework supports data binding).
 - Capture user input (e.g., through forms or actions).
- **Characteristics:**
 - Does not contain business logic; its sole purpose is to display information.
 - Often tightly coupled with the user interface design.
- **Example:**

In the same e-commerce application, a **View** could be a webpage displaying a product's details, such as its name, price, and availability, formatted in HTML and CSS.

3. Controller

Definition: The **Controller** acts as an intermediary between the **Model** and the **View**. It processes user input, updates the **Model**, and determines which **View** to display.

- **Key Responsibilities:**
 - Handle user actions (e.g., button clicks, form submissions).
 - Interpret user input and update the **Model** accordingly.
 - Decide which **View** to render and pass the necessary data to it.
- **Characteristics:**
 - Controls the application flow and user interaction.
 - Contains the logic to interpret requests and invoke appropriate responses.
- **Example:**

In the e-commerce application, the **Controller** might handle a "Buy Now" button click, update the **Model** (e.g., reduce inventory), and display a confirmation **View** with details about the order.

Workflow in the MVC Framework:

1. **User Interaction:** A user interacts with the **View** (e.g., clicks a button).
2. **Controller Processes Input:** The **Controller** captures the input, processes it, and updates the **Model**.
3. **Model Updates Data:** The **Model** applies the logic and updates the data.
4. **View Updates:** The updated data is displayed to the user through the **View**.

Benefits of MVC:

- **Separation of Concerns:** Each component focuses on a specific role, making the application easier to develop, test, and maintain.
- **Scalability:** Changes to the UI (View) can be made without affecting the business logic (Model).
- **Reusability:** Models and Views can be reused across different parts of the application.

Example Scenario:

Suppose you're building a **blog application**:

- **Model:** Manages data like blog posts, authors, and comments. E.g., `Post.getAllPosts()` retrieves all blog posts.
- **View:** Displays blog posts in a web page format, allowing users to read them.
- **Controller:** Handles requests, like fetching a specific post (`/post/{id}`), interacts with the **Model**, and sends the data to the **View** for rendering.

By dividing responsibilities, the MVC framework creates a clear structure that enhances development efficiency and collaboration.